War Event Forecasting

Python SaaS — Homework #2 Project Plan

Role	Name
Team Leader	Kovalov Evgen
Member	Tavlui Sofia
Member	Polyakov Kirilo
Member	Snikhovska Vladyslava
Member	Barabanova Sofia

Link	URL
GitHub Repo	https://github.com/exEKS/python_SaaS.git
Google Drive	

1. System Architecture Diagram

The system has three core modules — **Data Receiver**, **Forecasting**, and **Frontend (UI)** — with an optional shared storage layer. External data flows from APIs and web scraping through the data receiver, is transformed and modelled in the forecasting pipeline, and predictions are served via FastAPI and visualised in a Streamlit dashboard.

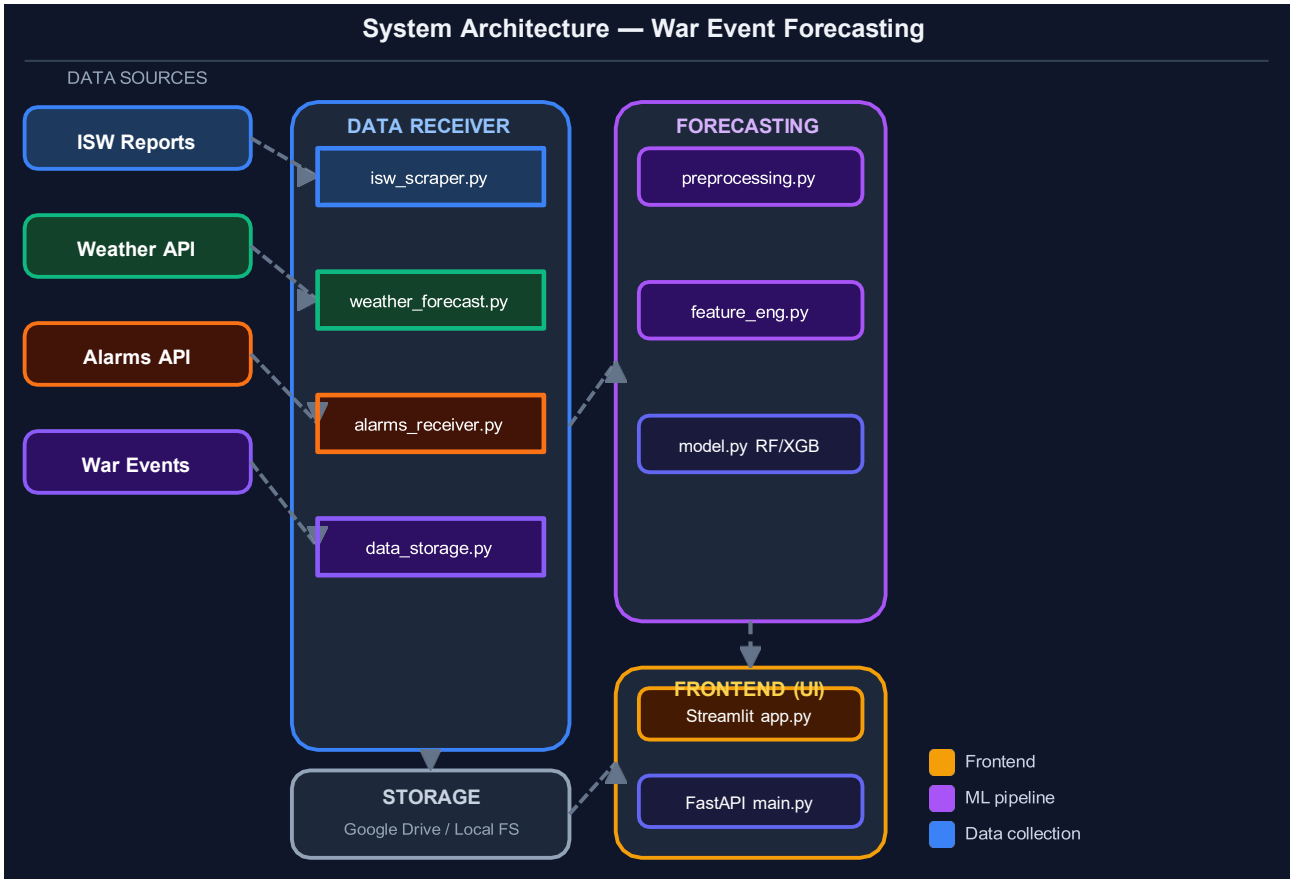

Figure 1. High-level system architecture. Dashed arrows indicate data flow.

2. Module Descriptions

2.1 Data Receiver

Collects all raw data from external sources on a daily schedule (APScheduler). Contains three sub-modules:

Script	Input	Output	Description
isw_scraper.py	Date range URL list	JSON {date, title, text}	Scrapes each daily ISW report from understandin gwar.org using requ
weather_forecast.py	Region coords API key	JSON: 24 h hourly forecast	Calls VisualCrossing REST API. Returns temp, p ecipitation, wind, hu
alarms_receiver.py	API key Region list	JSON: active alert snapshot	Polls devs.alerts.in.ua / api.ukrainealarm.com eve ry N minutes. Appe

2.2 Forecasting

Prepares features and trains/runs predictive models:

Script	Input	Output	Description
preprocessing.py	Raw text Weather CSV Alarm logs	Cleaned merged DataFrame	Strips HTML, tokenises ISW text, normalises wea ther numerics, align
feature_engineering.p	yCleaned DF	Feature matrix X Label vectors y	Adds lag features (t-1, t-2, t-7), 7-day rolling stats TF-IDF vectors (top
model.py	Feature matrix X Labels y	.pkl model files Prediction probs	Trains RandomForestClassifier and XGBClassifie r with 5-fold CV. Exp

2.3 Frontend (Streamlit + FastAPI)

Two-layer interface: FastAPI serves the model as a REST API; Streamlit provides an interactive dashboard.

Component	Input	Output	Description
FastAPI api/main.py	HTTP GET /predict?region= Kyiv&date=…	JSON: alarm_prob explosion_prob artillery_prob	Loads trained .pkl model, calls model.predict(), re turns JSON. Also ex
Streamlit frontend/app.py	User selects region + date	Interactive table + bar chart	Calls FastAPI /predict, renders probabilities with st.dataframe and st.b

3. Data Analysis & Preparation Strategy

3.1 Dataset Overview

Dataset	Period	Format	Est. Size
ISW daily reports	2022-02-24 – 2026-03-01	HTML -> text/JSON	~1 500 docs
Historical weather	2022-02-24 – 2026-03-01	CSV (hourly)	~35 000 rows
War events (alarms/expl.)	2022-02-24 – 2026-03-01	CSV (event log)	~300 000 rows
Real-time alarms	Rolling 24 h	JSON (API)	~500 rows/day

3.2 Text Data Preparation (ISW Reports)

ISW reports are the primary unstructured source. The pipeline below converts raw HTML articles into numeric feature vectors:

Step 1 — Collection: Scraper downloads each daily report; stores full text with {date, title, URL}.

Step 2 — Cleaning: HTML tags stripped. Boilerplate headers/footers removed with regex. Text lowercased; Ukrainian/Russian characters preserved. Non-alphabetic tokens (except region names) discarded.

Step 3 — Tokenisation: Text split on whitespace and punctuation. Custom stopword list (English + Ukrainian) applied.

Step 4 — Vectorisation: TF-IDF vectoriser (max 200 features, unigrams + bigrams) fitted on training corpus. Each report becomes a 200-dim sparse vector. Optional: sentence-transformers (paraphrase-multilingual-MiniLM) for 384-dim dense embeddings.

Step 5 — Feature alignment: Each vector assigned to publication date. Regions mentioned in the text receive a binary flag. Final row keyed on (date, region).

3.3 Structured Data Preparation

• Weather: Hourly data resampled to daily aggregates (avg/max/min) per region. Missing values forward-filled (max 3 h), then mean-imputed.

• Event logs: Alarm/explosion counts aggregated per (date, region, event_type). Lag features t-1, t-2, t-7 days and 7-day rolling mean added.

• Labels: Binary targets — alarm, explosion, artillery (1 = at least one event in next 24 h for that region).

• Train/test split: 80 % training (2022-02-24 – 2025-06-01), 20 % test (2025-06-01 – 2026-03-01). Temporal order preserved — no data leakage.

4. Team Responsibility List

Team Member	Tasks Completed
Kovalov Evgen	README.md System architecture diagram Module descriptions & I/O diagrams Data analysis plan & text preparation strategy Final PDF report compilation Code Review

** Screenshots of collected data, Postman API responses (VisualCrossing and Alarms API), and final GitHub / Google Drive links must be inserted by the responsible members before the 06/03/2026 deadline.*